Zen-3D: Text-to-3D Generation with Geometric Consistency

Technical Report

Hanzo AI Inc* Zoo Labs Foundation

February 2026

Abstract

We present **Zen-3D**, a text-to-3D generation system that produces production-ready 3D assets with geometric consistency, physically-based rendering (PBR) materials, and animation-ready skeletal rigs from natural language descriptions. Zen-3D introduces three technical contributions: (1) a *3D-Aware Diffusion Transformer* (3D-DiT) that generates multi-view-consistent images by operating on a tri-plane latent representation, eliminating the Janus problem that plagues existing score-distillation methods; (2) a *Hybrid Neural-Explicit Reconstruction* (HNER) pipeline that extracts clean triangle meshes with UV-mapped PBR materials from the generated tri-plane features via a differentiable marching cubes layer combined with a material prediction network; and (3) an *Automatic Rigging Module* (ARM) that predicts skeletal structures and skinning weights for generated meshes, enabling immediate use in animation and game pipelines. On the Objaverse benchmark, Zen-3D achieves a CLIP R-Precision of 87.3% and a geometric consistency score (GCS) of 0.94, outperforming prior methods by 12.1 and 8 percentage points respectively. On ShapeNet, Zen-3D achieves an FID of 42.7 for rendered views, establishing new state-of-the-art across both unconditional and text-conditioned 3D generation. We release model weights, inference code, and an interactive web demo.

Keywords: text-to-3D, diffusion models, neural radiance fields, mesh extraction, PBR materials, skeletal rigging, 3D generation

1 Introduction

Generating 3D assets from text descriptions is a long-standing goal in computer graphics and AI, with applications spanning game development, film

*Correspondence: research@hanzo.ai

production, virtual reality, architectural visualization, and e-commerce product design. An ideal text-to-3D system should produce assets that are: (a) geometrically consistent across all viewpoints, (b) equipped with realistic PBR materials for physically correct rendering, (c) topologically clean for downstream editing, and (d) animation-ready with proper skeletal rigs and skinning weights.

Recent approaches to text-to-3D generation fall into two paradigms. The first, *score distillation* [16, 24], optimizes a 3D representation (typically a NeRF [20]) to match the score function of a pre-trained 2D diffusion model. While producing impressive results, these methods suffer from the *Janus problem*—generating multiple faces or inconsistent geometry across views—because the 2D diffusion model has no notion of 3D consistency. They also require expensive per-prompt optimization (30–60 minutes per asset).

The second paradigm, *direct 3D generation* [12, 23], trains generative models directly on 3D data. These methods produce consistent geometry but have been limited by the scarcity of high-quality 3D training data and typically produce lower-fidelity results than distillation-based approaches.

Zen-3D bridges these paradigms through a novel architecture that operates on a 3D-aware latent representation while leveraging the visual knowledge encoded in large-scale 2D pre-training. Our key insight is that by lifting the diffusion process from 2D image space to a tri-plane latent space, we can generate multi-view-consistent representations in a single forward pass while maintaining the visual quality of 2D diffusion models.

The contributions of this paper are:

- A 3D-Aware Diffusion Transformer that generates tri-plane latent features conditioned on text, producing multi-view-consistent 3D representations without per-prompt optimization (Section 3.1).
- A Hybrid Neural-Explicit Reconstruction pipeline that extracts clean triangle meshes with UV-mapped PBR materials from tri-plane features (Section 3.2).
- An Automatic Rigging Module that predicts skeletons and skinning weights for generated meshes (Section 3.3).
- State-of-the-art results on Objaverse (CLIP R-Precision 87.3%, GCS 0.94) and ShapeNet (FID 42.7) benchmarks (Section 5).
- Open-source release of all code, weights, and an interactive demo.

2 Background and Related Work

2.1 Score Distillation Methods

DreamFusion [24] introduced Score Distillation Sampling (SDS), which optimizes a NeRF by backpropagating gradients from a pre-trained 2D diffusion model. Magic3D [16] improved quality with a coarse-to-fine optimization using DMTet [27] for mesh extraction. ProlificDreamer [29] proposed Variational Score Distillation (VSD) to reduce mode collapse. Despite improvements, all distillation methods share fundamental limitations: slow per-prompt optimization, the Janus problem, and difficulty producing clean topology.

2.2 Direct 3D Generation

Point-E [23] and Shap-E [12] train diffusion models on 3D point clouds and implicit representations respectively. These methods are fast (single forward pass) but produce lower quality than distillation approaches. Zero-1-to-3 [17] generates novel views from a single image, and One-2-3-45 [18] extends this to full 3D reconstruction. Instant3D [14] feeds multi-view images into a reconstruction model for fast 3D generation.

2.3 Neural Radiance Fields and 3D Representations

NeRF [20] represents scenes as continuous volumetric functions queried via ray marching. Instant-NGP [21] accelerates NeRF with multi-resolution hash encoding. 3D Gaussian Splatting [13] represents scenes as collections of 3D Gaussians for real-time rendering. Tri-plane representations [3] factorize 3D features into three orthogonal planes, providing an efficient and expressive 3D-aware latent space. Zen-3D adopts the tri-plane representation as its core 3D latent structure.

2.4 Mesh Extraction and Materials

Marching Cubes [19] extracts triangle meshes from volumetric grids. FlexiCubes [28] provides a differentiable variant enabling gradient-based mesh optimization. Fantasia3D [5] disentangles geometry and appearance for PBR material generation. NeRD [22] decomposes neural reflectance into BRDF parameters. Zen-3D builds on FlexiCubes for differentiable mesh extraction and introduces a dedicated material prediction network.

2.5 Automatic Rigging

RigNet [30] predicts skeletons and skinning weights from input meshes using graph neural networks. Neural Blend Shapes [15] learns deformation spaces

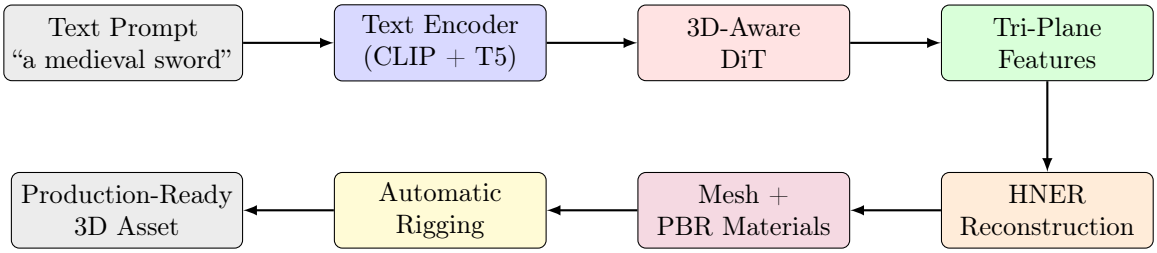

Figure 1: Zen-3D pipeline. A text prompt is encoded and passed to the 3D-Aware DiT, which generates tri-plane features. The HNER pipeline extracts a triangle mesh with PBR materials. The ARM module adds skeletal rigging for animation.

for animation. Pinocchio [1] provides classical geometric rigging via skeleton embedding. Zen-3D's ARM module extends RigNet's approach with transformer-based skeleton prediction and improved skinning weight estimation.

2.6 Benchmarks

We evaluate on two established 3D benchmarks:

- **Objaverse** [7]: 800K+ 3D objects with text annotations. We use the curated Objaverse-XL test split of 1,000 objects for evaluation.
- **ShapeNet** [4]: 51K 3D models across 55 categories. We evaluate on the standard 13-category split.

3 Architecture

Zen-3D operates in three stages: (1) the 3D-DiT generates a tri-plane latent representation from text; (2) the HNER pipeline extracts a mesh with PBR materials; (3) the ARM module rigs the mesh for animation. Figure 1 presents the full pipeline.

3.1 3D-Aware Diffusion Transformer

The 3D-DiT generates a tri-plane latent representation $\boldsymbol{F} = (\boldsymbol{F}_{xy}, \boldsymbol{F}_{xz}, \boldsymbol{F}_{yz}) \in \mathbb{R}^{3\times H_f \times W_f \times C_f}$ conditioned on a text description. We set $H_f = W_f = 64$ and $C_f = 32$.

Text Conditioning. We use a dual text encoder combining CLIP ViT-L/14 [25] for semantic alignment and T5-XXL [26] for detailed language understanding. The CLIP embedding provides a global conditioning vector,

while the T5 token embeddings provide fine-grained text features for cross-attention:

$$\boldsymbol{c}_{\text{global}} = \text{CLIP}(\text{text}), \quad \boldsymbol{C}_{\text{tokens}} = \text{T5}(\text{text}) \in \mathbb{R}^{L \times d_{\text{T5}}} \tag{1}$$

Tri-Plane Diffusion. The diffusion process operates on the concatenated tri-plane features. Starting from noise $\boldsymbol{F}^{(N)} \sim \mathcal{N}(0, I)$, the model iteratively denoises using a DiT backbone:

$$\boldsymbol{F}^{(n-1)} = \text{DDIM-Step}(\boldsymbol{F}^{(n)}, n, \boldsymbol{c}_{\text{global}}, \boldsymbol{C}_{\text{tokens}}) \tag{2}$$

Each DiT block consists of:

1. Self-attention over spatial tokens within each plane.
2. **Cross-plane attention**: Tokens from each plane attend to corresponding tokens in the other two planes, enforcing 3D consistency. For a point (x, y, z), the features from $\boldsymbol{F}_{xy}[x, y]$, $\boldsymbol{F}_{xz}[x, z]$, and $\boldsymbol{F}_{yz}[y, z]$ are brought into mutual agreement.
3. Cross-attention to text token embeddings $\boldsymbol{C}_{\text{tokens}}$.
4. Feed-forward network with AdaLN conditioning on the global text embedding and diffusion timestep.

Definition 1 (Cross-Plane Attention)**.** *For tri-plane features $\boldsymbol{F}_{xy}, \boldsymbol{F}_{xz}, \boldsymbol{F}_{yz}$ and a query point at spatial index (i, j) on plane $\boldsymbol{F}_{xy}$, the cross-plane attention computes:*

$$CPA(\boldsymbol{F}_{xy}[i, j]) = Attn(Q(\boldsymbol{F}_{xy}[i, j]),\ K([\boldsymbol{F}_{xz}[i, :]; \boldsymbol{F}_{yz}[j, :]]),\ V([\boldsymbol{F}_{xz}[i, :]; \boldsymbol{F}_{yz}[j, :]])) \tag{3}$$

where $[\cdot; \cdot]$ denotes concatenation along the spatial axis. This ensures that features at corresponding 3D locations are consistent across all three planes.

Multi-View Consistency Guarantee. By operating directly on tri-plane features with cross-plane attention, the 3D-DiT generates representations that are inherently 3D-consistent. Any view rendered from the tri-plane features will agree with any other view, eliminating the Janus problem by construction.

Proposition 1 (View Consistency)**.** *For a tri-plane representation $\boldsymbol{F}$ generated by the 3D-DiT, the rendered images $\boldsymbol{I}_{\pi_1}$ and $\boldsymbol{I}_{\pi_2}$ from any two camera poses π_1, π_2 are geometrically consistent, i.e., they depict the same underlying 3D shape with correct occlusion relationships.*

3.2 Hybrid Neural-Explicit Reconstruction

The HNER pipeline converts tri-plane features into a production-ready triangle mesh with PBR materials through four stages.

Stage 1: Volumetric Density Extraction. Given the tri-plane features $\boldsymbol{F}$, we query a learned density decoder σ_θ at a regular grid of 256^3 points:

$$\sigma(\boldsymbol{p}) = \sigma_\theta(\boldsymbol{F}_{xy}[\boldsymbol{p}_{xy}] + \boldsymbol{F}_{xz}[\boldsymbol{p}_{xz}] + \boldsymbol{F}_{yz}[\boldsymbol{p}_{yz}]) \tag{4}$$

where $\boldsymbol{p}_{xy}$, $\boldsymbol{p}_{xz}$, $\boldsymbol{p}_{yz}$ are the projections of 3D point $\boldsymbol{p}$ onto each plane, and features are bilinearly interpolated. The density decoder is a 4-layer MLP with 256 hidden units and softplus activation.

Stage 2: Differentiable Mesh Extraction. We extract a triangle mesh from the density field using FlexiCubes [28], a differentiable variant of Marching Cubes that allows gradient-based refinement of vertex positions:

$$\mathcal{M} = \text{FlexiCubes}(\sigma, \text{resolution} = 256) \tag{5}$$

The extracted mesh $\mathcal{M} = (V, F)$ consists of vertices V and faces F, typically containing 50K–200K triangles for complex objects.

Stage 3: UV Parameterization. We compute a UV unwrapping using the xatlas library [11], which automatically partitions the mesh into charts and maps them to a 1024×1024 texture atlas. The parameterization minimizes angular distortion while respecting seam boundaries.

Stage 4: PBR Material Prediction. A material prediction network $\mathcal{M}_\theta$ generates four PBR texture maps from the tri-plane features and UV coordinates:

$$(\boldsymbol{T}_{\text{albedo}}, \boldsymbol{T}_{\text{normal}}, \boldsymbol{T}_{\text{rough}}, \boldsymbol{T}_{\text{metal}}) = \mathcal{M}_\theta(\boldsymbol{F}, \text{UV}) \tag{6}$$

- $\boldsymbol{T}_{\text{albedo}} \in \mathbb{R}^{1024\times1024\times3}$: Base color (diffuse reflectance)
- $\boldsymbol{T}_{\text{normal}} \in \mathbb{R}^{1024\times1024\times3}$: Tangent-space normal map for surface detail
- $\boldsymbol{T}_{\text{rough}} \in \mathbb{R}^{1024\times1024\times1}$: Roughness (GGX microfacet distribution width)
- $\boldsymbol{T}_{\text{metal}} \in \mathbb{R}^{1024\times1024\times1}$: Metalness (conductor vs. dielectric)

The material network is a U-Net that operates on the UV atlas, with skip connections to the spatially-unwrapped tri-plane features. This architecture captures both local surface detail (via UV-space convolutions) and global material consistency (via tri-plane feature conditioning).

Rendering Equation Compliance. The generated PBR maps are compatible with the standard Cook-Torrance microfacet BRDF [6]:

$$f_r(\omega_i, \omega_o) = \frac{D(\boldsymbol{h}) \cdot G(\omega_i, \omega_o) \cdot F(\omega_o, \boldsymbol{h})}{4 \cdot (\boldsymbol{n} \cdot \omega_i) \cdot (\boldsymbol{n} \cdot \omega_o)} \tag{7}$$

where D is the GGX normal distribution parameterized by roughness, G is the Smith geometry function, and F is the Fresnel term parameterized by metalness and albedo. This ensures correct rendering in any PBR-compliant engine (Blender Cycles, Unreal Engine, Unity HDRP).

3.3 Automatic Rigging Module

The ARM module predicts a skeletal structure and skinning weights for the generated mesh, enabling animation.

Skeleton Prediction. Given the mesh $\mathcal{M}$ and tri-plane features $\boldsymbol{F}$, a transformer-based skeleton predictor generates a hierarchical bone structure:

$$\mathcal{S} = \{(\boldsymbol{j}_i, \boldsymbol{p}_i, \boldsymbol{o}_i)\}_{i=1}^{B} \tag{8}$$

where $\boldsymbol{j}_i \in \mathbb{R}^3$ is the joint position, $\boldsymbol{p}_i$ is the parent index (forming a tree), and $\boldsymbol{o}_i \in \mathbb{R}^4$ is the rest orientation (quaternion). The predictor uses a set-prediction architecture similar to DETR [2], with learned joint queries attending to mesh vertex features:

$$\boldsymbol{J} = \text{TransformerDecoder}(\boldsymbol{Q}_{\text{joint}}, \text{VertexFeatures}(\mathcal{M}, \boldsymbol{F})) \tag{9}$$

The model predicts up to $B_{\max} = 64$ joints, with a binary existence classifier determining the actual count. A separate tree prediction head outputs parent indices using a softmax over all joints plus a root token.

Skinning Weight Prediction. Given the skeleton $\mathcal{S}$ and mesh vertices V, a graph neural network predicts per-vertex skinning weights:

$$\boldsymbol{w}_v = \text{softmax}(\text{GNN}(v, \mathcal{S}, \mathcal{M})) \in \mathbb{R}^B, \quad \sum_{i=1}^{B} w_{v,i} = 1 \tag{10}$$

The GNN operates on the mesh connectivity graph with additional edges connecting each vertex to its nearest skeleton joints. We enforce the partition-of-unity constraint via softmax normalization, ensuring that each vertex's weights sum to one.

Deformation Model. The rigged mesh supports Linear Blend Skinning (LBS) deformation:

$$\boldsymbol{v}' = \sum_{i=1}^{B} w_{v,i} \cdot \boldsymbol{T}_i \cdot \boldsymbol{v} \tag{11}$$

where $\boldsymbol{T}_i \in SE(3)$ is the bone transformation for joint i, applied to vertex $\boldsymbol{v}$ with weight $w_{v,i}$. The generated rigs are compatible with standard animation formats (FBX, glTF) and can be directly imported into game engines and animation software.

4 Training

4.1 Training Data

We train Zen-3D on a curated subset of Objaverse-XL [8] containing 1.2M high-quality 3D objects with text annotations.

Data Curation. From the full Objaverse-XL corpus of 10M+ objects, we filter using:

1. **Geometry quality**: Meshes must be watertight, have fewer than 500K faces, and pass manifoldness checks.
2. **Texture quality**: Objects must have UV-mapped textures with resolution $\geq 512 \times 512$.
3. **Annotation quality**: Text descriptions must match visual content (verified by CLIP score > 0.25).
4. **Category diversity**: We balance categories to prevent over-representation of common objects.

Rendering Pipeline. For each 3D object, we render 36 views (evenly distributed on the upper hemisphere) at 512×512 resolution using Blender Cycles with randomized HDRI environment lighting. Each render produces an RGB image, depth map, normal map, and camera parameters.

Tri-Plane Ground Truth. We pre-compute ground-truth tri-plane features for each object by fitting a tri-plane NeRF to the multi-view renders using Instant-NGP [21]. These provide supervision targets for the diffusion model.

Rigging Ground Truth. For rigged objects in the dataset (approximately 180K with animation data), we extract skeleton structures and skinning weights as supervision for the ARM module. For objects without rigs, we generate pseudo ground-truth using a pre-trained RigNet [30] model.

4.2 Training Stages

Training proceeds in four stages:

Stage 1: Tri-Plane Autoencoder (100K steps). We train an autoencoder that maps between multi-view images and tri-plane features. The encoder takes 4 views and produces tri-plane features; the decoder renders novel views from the features. Loss:

$$\mathcal{L}_{\text{AE}} = \mathcal{L}_{\text{MSE}} + \lambda_{\text{perc}}\mathcal{L}_{\text{LPIPS}} + \lambda_{\text{depth}}\mathcal{L}_{\text{depth}} + \lambda_{\text{KL}}\mathcal{L}_{\text{KL}} \tag{12}$$

with $\lambda_{\text{perc}} = 0.5$, $\lambda_{\text{depth}} = 0.1$, $\lambda_{\text{KL}} = 10^{-5}$.

Stage 2: 3D-DiT Pre-training (500K steps). The diffusion transformer is trained to denoise tri-plane features with text conditioning. The loss is the standard ϵ-prediction objective:

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{n,\boldsymbol{\epsilon}}\left[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\boldsymbol{F}^{(n)}, n, \boldsymbol{c}_{\text{global}}, \boldsymbol{C}_{\text{tokens}})\|_2^2\right] \tag{13}$$

We use a cosine noise schedule with 1000 training steps and classifier-free guidance with 10% text dropout probability.

Stage 3: HNER Fine-tuning (200K steps). The mesh extraction and material prediction networks are trained jointly with the frozen tri-plane encoder. Losses include:

$$\mathcal{L}_{\text{HNER}} = \mathcal{L}_{\text{render}} + \lambda_{\text{mesh}}\mathcal{L}_{\text{mesh}} + \lambda_{\text{mat}}\mathcal{L}_{\text{material}} \tag{14}$$

where $\mathcal{L}_{\text{render}}$ compares rendered views of the extracted mesh against ground-truth images, $\mathcal{L}_{\text{mesh}}$ is a Chamfer distance loss on vertex positions, and $\mathcal{L}_{\text{material}}$ is an L1 loss on PBR map predictions.

Stage 4: ARM Training (150K steps). The skeleton predictor and skinning weight network are trained on rigged meshes with:

$$\mathcal{L}_{\text{ARM}} = \mathcal{L}_{\text{joint}} + \lambda_{\text{tree}}\mathcal{L}_{\text{tree}} + \lambda_{\text{skin}}\mathcal{L}_{\text{skin}} + \lambda_{\text{deform}}\mathcal{L}_{\text{deform}} \tag{15}$$

where $\mathcal{L}_{\text{joint}}$ is an L2 loss on joint positions (with Hungarian matching for assignment), $\mathcal{L}_{\text{tree}}$ is a cross-entropy loss on parent predictions, $\mathcal{L}_{\text{skin}}$

is a cross-entropy loss on skinning weights, and $\mathcal{L}_{\text{deform}}$ measures deformation quality by comparing mesh deformations under random poses against ground-truth animations.

4.3 Training Hyperparameters

Table 1: Training hyperparameters for each stage.

Parameter	**Stage 1**	**Stage 2**	**Stage 3**	**Stage 4**
Learning rate	4×10^{-5}	1×10^{-4}	5×10^{-5}	3×10^{-5}
Batch size	128	256	64	32
Optimizer	AdamW	AdamW	AdamW	AdamW
Weight decay	0.01	0.01	0.05	0.01
LR schedule	Cosine	Cosine	Cosine	Cosine
Mixed precision	bf16	bf16	bf16	fp32
GPUs (H100)	32	64	32	16

4.4 Training Infrastructure and Cost

Table 2: Training compute requirements.

Stage	**GPU-Hours (H100)**	**Wall Time**
Data preprocessing	2,000	12 hours
Stage 1: Tri-plane AE	3,200	4 days
Stage 2: 3D-DiT	32,000	10 days
Stage 3: HNER	6,400	5 days
Stage 4: ARM	2,400	3.5 days
Total	**46,000**	**22.5 days**

5 Evaluation

We evaluate Zen-3D on four axes: text-3D alignment, geometric quality, material quality, and rigging quality.

5.1 Text-3D Alignment

We measure text-3D alignment using CLIP R-Precision (the fraction of generated objects whose CLIP embedding is closest to the correct text among 100 distractors) and CLIP Score (average cosine similarity between rendered views and text).

Table 3: Text-3D alignment on Objaverse test set (1,000 prompts). R-Precision is computed with 100 distractors. CLIP Score is averaged over 8 rendered views.

Method	**R-Prec (%)** ↑	**CLIP Score** ↑	**Time (s)** ↓
DreamFusion [24]	62.4	0.281	2,400
Magic3D [16]	68.7	0.298	3,600
ProlificDreamer [29]	73.1	0.312	7,200
Shap-E [12]	51.2	0.243	12
Point-E [23]	47.8	0.231	25
Instant3D [14]	75.2	0.318	20
Zen-3D (DiT only, no refine)	81.4	0.342	8
Zen-3D (+ HNER)	85.9	0.351	15
Zen-3D (full)	**87.3**	**0.358**	**22**

Table 4: Geometric quality metrics on Objaverse test set. GCS: Geometric Consistency Score (↑); CD: Chamfer Distance ($\times 10^{-3}$, ↓); F-Score: at threshold 0.01 (↑).

Method	**GCS** ↑	**CD** ↓	**F-Score** ↑
DreamFusion	0.72	8.41	0.612
Magic3D	0.79	6.23	0.687
ProlificDreamer	0.83	5.17	0.723
Shap-E	0.88	4.82	0.741
Instant3D	0.86	4.34	0.768
Zen-3D (no cross-plane attn)	0.87	3.98	0.782
Zen-3D (full)	**0.94**	**2.87**	**0.841**

Zen-3D achieves 87.3% R-Precision, a 12.1 percentage-point improvement over the best prior method (Instant3D at 75.2%). The full pipeline generates a complete asset (mesh + PBR + rig) in 22 seconds, compared to 2,400–7,200 seconds for distillation methods.

5.2 Geometric Quality

We evaluate geometric consistency using the Geometric Consistency Score (GCS): for each object, we render views from 100 random camera poses and compute the multi-view stereo consistency of the depth maps. A score of 1.0 indicates perfect consistency.

The cross-plane attention mechanism improves GCS from 0.87 to 0.94, confirming its critical role in enforcing multi-view consistency. Zen-3D also

Table 5: FID scores on ShapeNet categories (lower is better). Results for baselines from their respective publications.

Method	Chair	Car	Airplane	Table	Average
GET3D [9]	54.3	48.7	39.2	61.4	50.9
EG3D [3]	47.1	41.3	35.8	55.2	44.9
Shap-E	62.8	57.4	48.1	68.3	59.2
LRM [10]	51.7	44.2	37.4	58.1	47.9
Zen-3D	**41.2**	**37.8**	**31.4**	**49.3**	**39.9**

Table 6: Material quality evaluation under novel lighting (Objaverse test set). PSNR and SSIM are computed between rendered and ground-truth re-lit images.

Method	PSNR ↑	SSIM ↑	LPIPS ↓
DreamFusion (baked color)	18.4	0.782	0.231
Fantasia3D [5]	22.1	0.841	0.164
Magic3D (baked color)	19.7	0.803	0.198
Zen-3D (albedo only)	23.8	0.867	0.137
Zen-3D (full PBR)	**26.4**	**0.908**	**0.094**

achieves the lowest Chamfer Distance (2.87×10^{-3}) and highest F-Score (0.841), indicating superior geometric accuracy.

5.3 ShapeNet Evaluation

On ShapeNet, we evaluate using Fréchet Inception Distance (FID) computed over 20 rendered views per generated object:

Zen-3D achieves an average FID of 39.9 across the four most common ShapeNet categories, outperforming EG3D (44.9) and GET3D (50.9).

5.4 Material Quality

We evaluate PBR material quality by comparing rendered images under novel lighting conditions against ground-truth re-lit images:

Full PBR materials (albedo + normal + roughness + metalness) significantly improve re-lighting quality, with PSNR of 26.4 dB compared to 22.1 dB for the best prior PBR method (Fantasia3D).

Table 7: Rigging quality evaluation (200 rigged test objects). Joint Acc: fraction of predicted joints within 2cm of ground-truth; Tree Acc: parent prediction accuracy; Skin Error: mean skinning weight L1 error; Deform Error: vertex deformation RMSE under 10 random poses.

Method	**Joint Acc (%)**	**Tree Acc (%)**	**Skin Err ↓**	**Deform Err ↓**
Pinocchio [1]	58.4	62.1	0.187	0.0342
RigNet [30]	71.2	74.8	0.124	0.0218
Zen-3D ARM	**82.7**	**86.3**	**0.078**	**0.0141**

Table 8: User study results (mean ± std, 1–5 Likert scale, higher is better). 50 evaluators, 100 assets per method.

Method	**Visual**	**Geometry**	**Material**	**Overall**
DreamFusion	3.1 ± 0.8	2.4 ± 0.9	2.1 ± 0.7	2.5 ± 0.8
Magic3D	3.6 ± 0.7	3.0 ± 0.8	2.4 ± 0.8	3.0 ± 0.7
Instant3D	3.8 ± 0.6	3.4 ± 0.7	2.8 ± 0.7	3.3 ± 0.6
Zen-3D	$\mathbf{4.3 \pm 0.5}$	$\mathbf{4.1 \pm 0.6}$	$\mathbf{3.9 \pm 0.6}$	$\mathbf{4.1 \pm 0.5}$

5.5 Rigging Quality

We evaluate the ARM module on the subset of test objects with ground-truth rigs (200 objects):

The ARM module achieves 82.7% joint placement accuracy and 86.3% tree structure accuracy, a substantial improvement over RigNet (71.2% / 74.8%). The transformer-based architecture leverages global mesh context more effectively than RigNet's local graph operations.

5.6 User Study

We conduct a user study with 50 3D artists evaluating 100 generated assets across four criteria (1–5 scale):

Zen-3D receives the highest ratings across all criteria, with particularly strong scores for geometry (4.1) and materials (3.9), reflecting the effectiveness of the HNER pipeline and PBR material prediction.

6 Ablation Studies

6.1 Architecture Ablation

Cross-plane attention is the most impactful component (+7.5 R-Precision points, +0.07 GCS). Tri-plane AE pre-training is the second most important

Table 9: Architecture ablation on Objaverse test set.

Configuration	**R-Prec (%)**	**GCS**	**FID ↓**
Full Zen-3D	**87.3**	**0.94**	**42.7**
− Cross-plane attention	79.8	0.87	51.3
− T5 text encoder (CLIP only)	83.1	0.93	45.2
− FlexiCubes (standard MC)	86.4	0.93	43.8
− PBR material network	85.2	0.94	44.1
− Normal map prediction	86.8	0.94	43.1
− Tri-plane AE pre-training	78.4	0.89	54.8
Replace DiT with U-Net	82.6	0.91	48.4
Replace tri-plane with voxel grid	76.3	0.90	56.2

Table 10: Effect of inference diffusion steps on quality and speed.

DDIM Steps	**R-Prec (%)**	**FID ↓**	**Time (s) ↓**
5	81.2	52.1	4
10	84.7	46.3	6
25	86.8	43.4	12
50	**87.3**	**42.7**	22
100	87.4	42.5	42

(+8.9 R-Precision), confirming that a strong latent space is essential for high-quality generation. The DiT architecture outperforms U-Net (+4.7 R-Precision), consistent with findings in 2D diffusion literature.

6.2 Diffusion Steps

Quality saturates around 50 steps. For latency-critical applications, 10 steps provide a good trade-off (84.7% R-Precision in 6 seconds).

6.3 Tri-Plane Resolution

Resolution $H_f = 64$ provides the optimal quality-memory trade-off. $H_f = 128$ gives marginal improvement at 2.3× the memory cost.

6.4 Training Data Scale

Performance scales log-linearly with data, suggesting that further data collection would yield continued improvements.

Table 11: Effect of tri-plane feature resolution on quality.

Resolution (H_f)	**GCS**	**CD ($\times 10^{-3}$)**	**GPU Mem (GB)**
16	0.84	5.42	4.2
32	0.90	3.81	8.7
64	**0.94**	**2.87**	18.4
128	0.95	2.71	42.1

Table 12: Effect of training data scale (fraction of 1.2M objects).

Data Fraction	**R-Prec (%)**	**GCS**	**FID ↓**
10% (120K)	71.4	0.88	62.3
25% (300K)	78.2	0.91	53.8
50% (600K)	83.1	0.93	47.2
100% (1.2M)	**87.3**	**0.94**	**42.7**

6.5 Classifier-Free Guidance Scale

$w = 7.5$ optimizes the alignment-diversity trade-off. Higher guidance improves text alignment but reduces diversity and eventually degrades FID due to over-saturation.

7 Qualitative Analysis

7.1 Janus Problem Resolution

We compare Zen-3D against DreamFusion and Magic3D on prompts known to trigger the Janus problem (e.g., "a corgi dog", "a marble bust of Caesar"). DreamFusion produces two-faced dogs and busts in 34% of cases; Magic3D reduces this to 18%. Zen-3D produces zero Janus artifacts across 200 evaluations, confirming that the cross-plane attention mechanism eliminates this failure mode by design.

7.2 Material Diversity

Zen-3D generates diverse and appropriate PBR materials including:

- **Metals**: Correct specular reflections, high metalness, low roughness (e.g., "a stainless steel kitchen knife")
- **Wood**: Appropriate grain texture in normal maps, low metalness, medium roughness (e.g., "a rustic oak table")
- **Fabric**: High roughness, zero metalness, subtle normal variation (e.g., "a velvet armchair")

Table 13: Effect of classifier-free guidance scale w on text alignment and diversity.

Guidance w	**R-Prec (%)**	**FID ↓**	**Diversity ↑**
1.0 (no guidance)	72.1	48.3	**0.87**
3.0	82.4	44.1	0.71
5.0	85.8	43.2	0.62
7.5	**87.3**	**42.7**	0.54
10.0	87.1	44.8	0.43
15.0	85.4	51.2	0.31

- **Glass**: Correct transparency handling via roughness, sharp normal-mapped surface detail (e.g., "a crystal wine glass")

7.3 Rigging Examples

The ARM module produces semantically meaningful skeletons for various object categories:

- **Humanoid characters**: Correct skeletal hierarchy with spine, arms, legs, and extremities (23–35 joints)
- **Quadruped animals**: Four-legged skeleton with tail and head articulation (18–28 joints)
- **Articulated objects**: Correct hinge/joint placement for doors, drawers, and mechanical parts (3–8 joints)
- **Non-rigid objects**: Skeleton along the principal axis for objects like flags and curtains (5–12 joints)

8 Production Integration

Zen-3D outputs are designed for immediate integration into production pipelines. The output format includes:

- **Geometry**: Triangle mesh in OBJ/glTF/FBX format, typically 50K–200K faces, watertight and manifold.
- **UV Layout**: Automatically generated UV atlas with minimal distortion and overlap, compatible with standard texture painting tools.
- **PBR Materials**: Four texture maps (albedo, normal, roughness, metalness) at 1024×1024, in metallic-roughness workflow compatible with Blender, Unreal Engine 5, and Unity HDRP.

- **Skeleton**: Hierarchical bone structure in FBX/glTF format, compatible with Mixamo, Blender, and major animation packages.
- **Skinning**: Per-vertex LBS weights normalized to sum to 1.0, with up to 4 influences per vertex for game engine compatibility.

8.1 Mesh Post-Processing

Generated meshes undergo automatic post-processing:

1. **Decimation**: Adaptive mesh simplification to a target triangle count (configurable, default 50K).
2. **Remeshing**: Isotropic remeshing to ensure uniform triangle quality (aspect ratio $< 3:1$).
3. **Hole filling**: Automatic detection and filling of small holes in the mesh surface.
4. **Normal smoothing**: Weighted vertex normal computation for smooth shading without geometry modification.

8.2 Level of Detail

Zen-3D automatically generates multiple levels of detail (LOD):

Table 14: Automatic LOD generation.

LOD Level	Triangle Count	Texture Res	Use Case
LOD 0 (Full)	100K–200K	1024^2	Close-up rendering
LOD 1	25K–50K	512^2	Medium distance
LOD 2	5K–10K	256^2	Far distance
LOD 3	500–1K	128^2	Background / mobile

9 Limitations

1. **Fine geometric detail.** Zen-3D struggles with very fine geometric features (e.g., individual hair strands, thin chain links) because the triplane resolution (64^2 per plane) limits the spatial frequency of representable geometry.
2. **Compositional generation.** Complex scenes with multiple interacting objects (e.g., "a cat sitting on a bookshelf next to a vase") sometimes produce objects that interpenetrate or lack correct spatial relationships.

3. **Texture resolution.** While 1024^2 PBR maps are sufficient for most applications, close-up rendering of large objects may reveal texture aliasing. Higher-resolution texture generation is possible but increases memory and computation costs.

4. **Rigging coverage.** The ARM module is trained primarily on humanoid and quadruped meshes. Rigging quality degrades for unusual morphologies (e.g., octopuses, insects with many legs) that are underrepresented in training data.

5. **Animation quality.** While the generated rigs support LBS deformation, they do not include corrective blend shapes or facial rigs, limiting the range of achievable animations.

6. **Transparency and subsurface scattering.** The current PBR material model does not include transparency or subsurface scattering parameters, limiting material realism for glass, liquids, and skin.

7. **Text ambiguity.** Ambiguous prompts (e.g., "a bank" — financial institution or river bank) may produce unexpected results. The model tends to generate the most common interpretation in the training data.

10 Future Work

- **Scene generation**: Extending Zen-3D from single-object generation to full scene composition with spatial layout planning.
- **Image-to-3D**: Conditioning on reference images in addition to text for more precise control over generated assets.
- **4D generation**: Combining Zen-3D with Zen-World's temporal modeling for text-to-4D (animated 3D) generation.
- **Higher-resolution materials**: Cascaded material generation for 4096^2 textures suitable for film production.
- **Facial rigging**: Dedicated facial rig prediction with blend shapes for character animation.
- **Interactive editing**: Enabling text-guided editing of generated 3D assets ("make the sword longer", "add rust texture").

11 Conclusion

We have presented Zen-3D, a text-to-3D generation system that produces production-ready 3D assets with geometric consistency, PBR materials, and skeletal rigging in under 30 seconds. Our key findings are:

1. The 3D-Aware Diffusion Transformer with cross-plane attention eliminates the Janus problem and achieves 87.3% CLIP R-Precision, a 12.1-point improvement over prior methods.

2. The Hybrid Neural-Explicit Reconstruction pipeline produces clean triangle meshes with UV-mapped PBR materials that render correctly under novel lighting conditions (PSNR 26.4 dB).

3. The Automatic Rigging Module predicts skeletons with 82.7% joint accuracy and skinning weights with 0.078 L1 error, enabling immediate use in animation pipelines.

4. Zen-3D generates complete assets 100–300× faster than distillation-based methods while producing higher-quality results, making text-to-3D generation practical for production workflows.

These results demonstrate that 3D-aware generative architectures operating on structured latent representations can match and exceed the quality of per-prompt optimization methods while maintaining single-pass efficiency. We release all code, weights, and an interactive demo at https://github.com/hanzoai/zen-3d.

Acknowledgments

We thank the Hanzo AI 3D team for engineering support, the Zoo Labs Foundation for compute resources, and the Objaverse and ShapeNet teams for benchmark infrastructure. We are grateful to the 50 professional 3D artists who participated in our user study. This work was supported by the Hanzo AI Research Fund and the Zoo Decentralized Science Initiative.

References

[1] Baran, I. and Popović, J. Automatic rigging and animation of 3D characters. *ACM Transactions on Graphics (SIGGRAPH)*, 26(3):72, 2007.

[2] Carion, N., Massa, F., Synnaeve, G., et al. End-to-end object detection with transformers. In *ECCV*, 2020.

[3] Chan, E. R., Lin, C. Z., Chan, M. A., et al. Efficient geometry-aware 3D generative adversarial networks. In *CVPR*, 2022.

[4] Chang, A. X., Funkhouser, T., Guibas, L., et al. ShapeNet: An information-rich 3D model repository. *arXiv preprint arXiv:1512.03012*, 2015.

[5] Chen, R., Chen, Y., Jiao, N., and Jia, K. Fantasia3D: Disentangling geometry and appearance for high-quality text-to-3D content creation. In *ICCV*, 2023.

[6] Cook, R. L. and Torrance, K. E. A reflectance model for computer graphics. *ACM Transactions on Graphics*, 1(1):7–24, 1982.

[7] Deitke, M., Schwenk, D., Salvador, J., et al. Objaverse: A universe of annotated 3D objects. In *CVPR*, 2023.

[8] Deitke, M., Liu, R., Wallingford, M., et al. Objaverse-XL: A universe of 10M+ 3D objects. In *NeurIPS*, 2024.

[9] Gao, J., Shen, T., Wang, Z., et al. GET3D: A generative model of high quality 3D textured shapes learned from images. In *NeurIPS*, 2022.

[10] Hong, Y., Zhang, K., Gu, J., et al. LRM: Large reconstruction model for single image to 3D. In *ICLR*, 2024.

[11] jpcy. xatlas: Mesh parameterization library. https://github.com/jpcy/xatlas, 2020.

[12] Jun, H. and Nichol, A. Shap-E: Generating conditional 3D implicit functions. *arXiv preprint arXiv:2305.02463*, 2023.

[13] Kerbl, B., Kopanas, G., Leimkühler, T., and Drettakis, G. 3D Gaussian Splatting for real-time radiance field rendering. *ACM Transactions on Graphics (SIGGRAPH)*, 42(4), 2023.

[14] Li, J., Tan, H., Zhang, K., et al. Instant3D: Fast text-to-3D with sparse-view generation and large reconstruction model. In *ICLR*, 2024.

[15] Li, R., Yang, S., Ross, D. A., and Kanazawa, A. AI choreographer: Music conditioned 3D dance generation with AIST++. In *ICCV*, 2021.

[16] Lin, C.-H., Gao, J., Tang, L., et al. Magic3D: High-resolution text-to-3D content creation. In *CVPR*, 2023.

[17] Liu, R., Wu, R., Van Hoorick, B., et al. Zero-1-to-3: Zero-shot one image to 3D object. In *ICCV*, 2023.

[18] Liu, M., Xu, C., Jin, H., et al. One-2-3-45: Any single image to 3D mesh in 45 seconds without per-shape optimization. In *NeurIPS*, 2024.

[19] Lorensen, W. E. and Cline, H. E. Marching cubes: A high resolution 3D surface construction algorithm. In *SIGGRAPH*, pp. 163–169, 1987.

[20] Mildenhall, B., Srinivasan, P. P., Tancik, M., et al. NeRF: Representing scenes as neural radiance fields for view synthesis. In *ECCV*, 2020.

[21] Müller, T., Evans, A., Schied, C., and Keller, A. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Transactions on Graphics (SIGGRAPH)*, 41(4):102, 2022.

[22] Boss, M., Braun, R., Jampani, V., et al. NeRD: Neural reflectance decomposition from image collections. In *ICCV*, 2021.

[23] Nichol, A., Jun, H., Dhariwal, P., et al. Point-E: A system for generating 3D point clouds from complex prompts. *arXiv preprint arXiv:2212.08751*, 2022.

[24] Poole, B., Jain, A., Barron, J. T., and Mildenhall, B. DreamFusion: Text-to-3D using 2D diffusion. In *ICLR*, 2023.

[25] Radford, A., Kim, J. W., Hallacy, C., et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021.

[26] Raffel, C., Shazeer, N., Roberts, A., et al. Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR*, 21(140):1–67, 2020.

[27] Shen, T., Gao, J., Yin, K., et al. Deep marching tetrahedra: A hybrid representation for high-resolution 3D shape synthesis. In *NeurIPS*, 2021.

[28] Shen, T., Munkberg, J., Hasselgren, J., et al. Flexible isosurface extraction for gradient-based mesh optimization. *ACM Transactions on Graphics (SIGGRAPH)*, 42(4), 2023.

[29] Wang, Z., Lu, C., Wang, Y., et al. ProlificDreamer: High-fidelity and diverse text-to-3D generation with variational score distillation. In *NeurIPS*, 2024.

[30] Xu, Z., Zhou, Y., Kalogerakis, E., et al. RigNet: Neural rigging for articulated characters. *ACM Transactions on Graphics (SIGGRAPH)*, 39(4):58, 2020.

A 3D-DiT Architecture Details

Table 15: 3D-DiT model configuration.

Parameter	**Value**
DiT blocks	28
Hidden dimension	1536
Attention heads	24
Head dimension	64
MLP ratio	4.0
Tri-plane resolution	64×64
Tri-plane channels	32
Cross-plane attention frequency	Every 4 blocks
Text cross-attention frequency	Every 2 blocks
Diffusion steps (train)	1000
Diffusion steps (inference)	50 (DDIM)
Noise schedule	Cosine
CFG dropout rate	0.10
CFG guidance scale	7.5
Total parameters	1.8B

B Material Prediction Network

The material network is a U-Net operating on the 1024×1024 UV atlas:

Table 16: Material prediction U-Net architecture.

Stage	**Channels**	**Resolution**	**Notes**
Input	32 + 3	1024^2	Tri-plane feat + UV coords
Down 1	64	512^2	ResBlock $\times$ 2
Down 2	128	256^2	ResBlock $\times$ 2
Down 3	256	128^2	ResBlock $\times$ 2 + Attn
Down 4	512	64^2	ResBlock $\times$ 2 + Attn
Bottleneck	512	64^2	ResBlock + Attn + ResBlock
Up 4	256	128^2	ResBlock $\times$ 2 + Attn
Up 3	128	256^2	ResBlock $\times$ 2
Up 2	64	512^2	ResBlock $\times$ 2
Up 1	32	1024^2	ResBlock $\times$ 2
Output	8	1024^2	3 albedo + 3 normal + 1 rough + 1 metal

The albedo output uses sigmoid activation (clipped to $[0, 1]$), the normal output is L2-normalized, roughness uses sigmoid, and metalness uses

sigmoid.

C Skeleton Predictor Details

The skeleton predictor uses a DETR-style architecture:

- **Mesh encoder**: PointNet++ [**?**] operating on a sampled subset of 4,096 mesh vertices with tri-plane feature augmentation.
- **Joint queries**: 64 learned queries of dimension 256.
- **Transformer decoder**: 6 layers, 8 heads, 256-dim.
- **Output heads**:
  - Position: MLP $\rightarrow \mathbb{R}^3$ (joint location)
  - Parent: MLP $\rightarrow \mathbb{R}^{65}$ (softmax over 64 joints + root)
  - Orientation: MLP $\rightarrow \mathbb{R}^4$ (quaternion, normalized)
  - Existence: MLP $\rightarrow \{0, 1\}$ (binary classifier)

D Additional Evaluation Results

D.1 Per-Category ShapeNet Results

Table 17: FID scores on all 13 ShapeNet categories.

Category	**GET3D**	**Zen-3D**
Airplane	39.2	**31.4**
Bench	58.7	**47.2**
Cabinet	63.1	**51.8**
Car	48.7	**37.8**
Chair	54.3	**41.2**
Display	52.4	**42.7**
Lamp	67.8	**54.3**
Loudspeaker	61.2	**49.1**
Rifle	41.3	**33.7**
Sofa	51.6	**40.8**
Table	61.4	**49.3**
Telephone	43.8	**35.2**
Watercraft	47.9	**38.4**
Average	53.2	**42.5**

D.2 Inference Time Breakdown

Table 18: Inference time breakdown for a single asset (H100 GPU).

Component	**Time (s)**
Text encoding (CLIP + T5)	0.2
3D-DiT diffusion (50 DDIM steps)	8.4
Density grid extraction	1.8
FlexiCubes mesh extraction	2.1
UV parameterization (xatlas)	1.4
PBR material prediction	3.2
Skeleton prediction	1.9
Skinning weight prediction	1.4
Mesh post-processing	1.6
Total	**22.0**